

Source code style and indentation guidelines

R&D Squad
V4

General tips

- ✗ All comments, variable names, functions names and etc. must be in English.
- ✗ If you need more than 3 levels of indentation, you're screwed and you should fix your program.
- ✗ The limit on the length of lines is 120 columns and this is a strongly preferred limit.
- ✗ Tab length is 4 spaces always. Don't use tabs, use 4 spaces.
- ✗ It is preferred that the line width does not exceed the width of the Comment Blocks in order to keep the code readable on screen and on the printer. In some cases however it is difficult to avoid
- ✗ Don't use floating-point variables where discrete values are needed. Using a float for a loop counter is a great way to run onto problems. Always test floating-point numbers as $\leq$ or $\geq$, never use an exact comparison ($==$ or $!=$).
- ✗ When testing increment or decrement counters try to use the $\geq$ or $\leq$ instead of $==$ in if-statements. This is one of the important defensive programming rules. If for some reason the value of the counter gets higher or lower than expected in the test for equal you would run into problems.
- ✗ *.c and *.h files is named like this: *my_c_file.c*; *my_h_file.h*. Use lower case characters and separate words with “_”.
- ✗ Functions must do one thing and only one thing!
- ✗ Avoid using global variables. Instead, pass variables by reference to functions that change their value. Or if you need to use it, use it wisely, always consider that context switch, interrupt can occur. In such case it's good practice to disable and enable global interrupts if that variable is used in different contexts or interrupts.
- ✗ Avoid using function variables with *static* attribute in multithread systems or at least be aware of that. Always have in mind that function might be called in different threads at the same time, and if you have *static* variable, it could lead to undefined behavior.
- ✗ No magic numbers! Define constants and use them in your program rather than using numerical values. Constants make your code more readable, and easier to change. Also don't forget to add comment for constant.
- ✗ All functions (private and exported), variables (private, local and exported) must be written in accordance with this format: **this_is_my_var_func_name**. No capital letters and joined words are allowed. For example this is not correct: **THIS_is_MyVar_func_Name**.
- ✗ Do not use *unsigned int*, *unsigned short* and etc. variable types, use *uint32_t*, *int32_t*, *uint8_t*, *int16_t* and so on instead. For pointers and char arrays *char[]* or *char** can be used. But remember that *char* is from -128 to 127. Better not to use *char* for counting or for index number.
- ✗ Number of arguments passed to a function should not be greater than 4. 1 – ok, 2 – should be ok, 3 – also ok, 4 – if you have no other choice.
- ✗ When you pass a buffer (pointer to an array) it's good practice to also pass buffer size. This way, function would know what is the size of buffer and could act accordingly. For example when filling a buffer, function

would know when to stop.

- ✗ If you write a function which takes argument as a pointer, and in function that argument will not be modified, declare that argument (variable) as constant.
For example: **void uart_send_data(const uint8_t *data, uint32_t size);**
- ✗ When you pass a pointer through several function (deeper and deeper), and other functions are static or can't be reached to outside world, there is no need to check that pointer in every function, only in the highest one (by call hierarchy).
- ✗ Avoid using *bool* function arguments. If you have such argument, it means that you had to write two functions for each case (*true* and *false*) instead of one.
- ✗ Avoid adding variable name prefix that describes what variable type is, for example: *uint8_t *p_data*. We use modern IDE's, like Eclipse which can quickly tell as what variable type is. Variable name should describe for what purpose it's used. The same rule should apply for functions names.
- ✗ It's a good practice in function to check errors at the beginning and return the error code, or just that it failed (*false*).

Modules explanation

H file

Includes

Under the section **Includes** all modules that are used or needed have to be included using the `#include` statement. When standard C-header files (example `#include`) are used then these will have to be included first. After this, separated by an empty line, the include section follows for the local include files (example `#include "lis3dh_api.h"`). It is recommended to use an alphabetical ordering on the modules included, so that it not only looks better, but also allows for quickly finding out if a specific module is used or not.

```

/*****
/* Includes */
*****/
#include <stdint.h>

#include "lis3dh_api.h"
#include "spi_driver.h"

```

Exported definitions and macros

All definitions (`#define`) and macro's have to be placed under the **Exported definitions and macros** section. This can be done in both the header and source file. In which file the definition or macro appears will depend on it's scope. Definitions or macro's which have to be known outside the modules (thus exported) will have to be put in the header file. A definition is everything that is defined using a `#define`. A macro is a definition that uses arguments. To allow for a good recognition of defines and macro's it is recommended to use capitals completely. To increase readability in this case it is allowed to use underscores. Logical groups of defines and macro's have to be ordered together. All defines use the same indentation columns starting at the left margin. The different groups can be separated by using proper comment blocks and empty lines.


```

/
*****/
/* Exported definitions and macros */
*****/
#define TEMPLATE_CONSTANT_NAME 1 //!< Defines constant name length
#define TEMPLATE_COUBTER_MAX 5 //!< This number defines counter maximum value

#define TEMPLATE_SUM(A, B) (A + B) //!< This macros sums two variables.
#define TEMPLATE_BIT_SET(A, B) ((A) |= (1 << (B))) //!< This macros sets corresponding bit.

```

Exported types

A type definition have to be placed under the **Exported types** section. It has to meet a number of rules:

- The name of the **typedef** starts with template name or acronym of that name(e.g. if C or H file name is **data_manager.c** beginning of name will be **dmngr_** – use acronym to avoid long names – should be not more than 12 symbols). Moreover, acronyms should be logic and understandable – use brainZ captain! :)
- Name should end with ‘_t’ - aka Type.
- Words in name has to be separated by underscores(e.g. **dmngr_timer0_cfg_opt_t**) to improve readability.
- Every enum unit starts with file name in upper cases "**TEMPLATE_**" or its acronym "**TMPLT_**" and rest is also in upper cases "**TEMPLATE_ENUM_1**".
- For structure **typedef** name the same logic is applied.

```

*****/
/* Exported types */
*****/
typedef enum
{
    TEMPLATE_ENUM_1, //!< TEMPLATE_ENUM_1
    TEMPLATE_ENUM_2, //!< TEMPLATE_ENUM_2
    TEMPLATE_ENUM_3, //!< TEMPLATE_ENUM_3
} template_enum_t;

typedef struct
{
    uint8_t variable_name;
    uint16_t variable_name_2; /*!< This variable is my favorite it hold all the happenings in the world
                                and even more. Please use it carefully. */
} template_struct_t; /*!< This ... See @ref template_enum_t. */

```

Prototypes of exported variables and constants

In this section all exported variables from the module have to be declared between the compiler directives using the keyword **extern**. The declared variables in the exported section have to be declared in the C source file as global variables under the same section and without the static keyword in order to export it's scope to the outside world.

```

*****/
/* Prototypes of exported variables and constants */
*****/
extern const template_key_code_t template_key_table [];
extern uint8_t nr_of_key_codes;

```

Prototypes of exported functions

In this section the exported functions of the modules are placed with their function prototypes. For each exported function a comment block has to be added above the function prototype that gives a good description of the meaning and use of the functions and it's arguments.

```

/*****
/* Prototypes of exported functions */
*****/

/**
 * @brief Brief
 *
 * @return Return_Description
 *
 * @details Details
 */
uint16_t ekb2_init (void);

/**
 * @brief Brief
 *
 * @param id      Parameter_Description
 * @param data    Parameter_Description
 * @return Return_Description
 *
 * @details Details
 */
void template_function(uint8_t id, uint8_t *data);

```

C File

Includes

Refer to the Header File Template paragraph for a detailed description on this section.

Private definitions and macros

Refer to the Header File Template paragraph for a detailed description on this section

Private types

Refer to the Header File Template paragraph for a detailed description on this section.

Private variables and constants

Static declared global variables must be placed under the section **Private variables and constants** in the source file. The variable names should be placed one below the other, preferably left aligned in the same column and by preference on the same indent position. Names must start with module name or its acronym. The declaration should start at the left margin. The scope of these variables is global to the source file and are not known to the outside world due to the static keyword.

```

/*****
/* Private variables and constants */
*****/

static template_stack_t template_idle_stack[STACKS_IDLE_STACK_SIZE];

```

```
static uint16_t template_rr_count;
static uint16_t template_preemption_count;
static uint16_t template_ready_q;
static uint16_t template_sleep_q;
```

Exported variables and constants

In this section all exported variables from the module have to be declared without using the keywords *static* or *extern*. By omitting the static keyword the variable is exported or public. Names must start with module name or its acronym.

```
/* Exported variables and constants */
uint8_t template_active_task;
uint8_t template_old_task;
```

Prototypes of local functions

In this section the prototypes of the local functions of the source file are placed. All local functions and their prototypes have the keyword *static* in order to keep the scope ‘inside’ (not exported) to the source file. Names must start with module name or its acronym. Note however that the functions are globally accessible from within the source file.

```
/* Prototypes of local functions */
static void template_kernel_tick_handler(void);
static void template_reschedule(void);
static void template_ready(uint8_t tid, uint8_t action);
static void template_status_ready(uint8_t tid, uint8_t action);
```

Private functions

The function declaration and the implementation of a local function must be placed under this section in the source file. All functions appearing in this section must have a corresponding function prototype in the section **Prototypes of local functions**.

```
/* Private functions */
static void template_status_ready(uint8_t tid, uint8_t action)
{
    ...
}
```

Exported functions

In this section the implementations of the exported functions of the module are placed.

```
/* Exported functions */
bool template_timer_init(void)
{
    ...// Your code.
}
```



Coding style related

Switch-case statement

```

/*
 * switch - prototype
 * 1. Every case value should be predefined value using enumerators.
 * 2. Must always have "default" case.
 * 3. Every case must be tabbed by 4 spaces from virtual "switch" vertical line.
 */
switch(var3)
{
....case TEMPLATE_ENUM_1:
....{
.....// Your code.
.....break;
....}
....case TEMPLATE_ENUM_2:
....{
.....// Your code.
.....break;
....}
....default:
....{
.....// Your code.
.....break;
....}
}

```

FOR cycle

```

/*
 * for - prototype
 * Variables that is used for counting, should be one letter - 'i', 'j'.
 */
for(i = 0; i < TEMPLATE_COUNTER_MAX; i++)
{
....// Your code.
}

```

IF conditional statements

```

/*
 * if - prototype
 * 1. Every variable and operator should be separated by one space ' '.
 * 2. In case of multiple conditions:
 *   - Conditions must be aligned to left by first's condition variable
 *   - Conditions must be in braces
 *   - Operators must placed on right side after every condition
 * 3. Curly braces around if/else block is a must if in if or else block there are more than one line of code.
 */
if( (template_var1 == var2) &&
    (template_var2 == var3) &&
    (template_long_variable == true) )
{
....// Your code.
}
else
{
....// Your code.
}

```


While loop

```
/*  
 * while - prototype  
 */  
while(1)  
{  
    ....// Your code.  
}
```

Do-while loop

```
/*  
 * do while – prototype  
 * While should be in the same line as closing do curly brace “}”.  
 */  
do  
{  
    ....// Your code.  
} while(var1 < var2);
```



Code formatting

Space for tabs

4 spaces match 1 TAB. And must be inserted instead of TAB. Configure this in Keil uVision IDE:
Edit → Configuration → Editor → In “C/C++ Files”, “ASM Files” and “Other Files” select “Insert spaces for tabs”
checkbox and set value to 4

Practice functional programming

Ref: example_practical_c.c

General tips.....2

Modules explanation.....2

H file.....2

C File.....4

Coding style related.....6



[Switch-case statement.....6](#)

[FOR cycle.....6](#)

[IF conditional statements.....7](#)

[While loop.....7](#)

[Do-while loop.....7](#)

[Code formatting.....7](#)

[Space for tabs.....7](#)

[Software.....7](#)

[Usage & Configuration.....7](#)

[How to use.....8](#)

[Practice functional programming.....8](#)